



CARRERA DE SOFTWARE “PATRONES DE SOFTWARE” TRABAJO: GRUPAL

Fecha máxima de presentación: una semana antes del final del semestre. Si se presenta luego de esta fecha se penalizará con 1 punto menos por cada día transcurrido (contando el fin de semana).

NOTA: El proyecto es la base del examen del segundo parcial.

Procedimiento:

- Se debe inicialmente **seleccionar** uno de los trabajos presentados (la factura individual del primer parcial) para el proyecto final grupal.
- Realizar el desarrollo del proyecto utilizando una **arquitectura de software** (por ejemplo, **programación por capas con clean architecture**), buenas prácticas de programación, código limpio, principios SOLID y patrones de Software donde sea necesario sin hacer sobre ingeniería, para obtener software de calidad, que sea fácil de entender, facilitar el trabajo en grupo, sea escalable, simplificar el mantenimiento y el control de errores, que sea fácil de testear.
- El proyecto debe **implementar obligatoriamente** una capa de servicios con **REST (Web API)**, esta capa es la única que el front-end debe consumir.

PUNTO DE VENTA (WEB) – NO ES UN CARRITO DE COMPRAS

Un **orden de venta** es el nombre genérico que se da al software utilizado **para** recoger los pedidos de sus clientes (FACTURACIÓN). La función de este software es permitir realizar la facturación vía una aplicación (aplicación web en este caso), de todos los artículos (productos, inventario) que desean comprar los clientes. Cuando el cliente ha seleccionado físicamente los productos deseados, puede pasar a la "caja-punto de venta" y pagar para que se proceda al proceso de la FACTURACIÓN.

Descripción del proyecto

Se pide entonces desarrollar un sistema web de punto de venta para registrar ventas/facturas en un micromercado, supermercado pequeño o negocio similar.

El sistema no corresponde a un carrito de compras. La venta se realiza desde una interfaz de caja donde el vendedor selecciona productos, valida stock, calcula totales y confirma la venta.

El frontend debe consumir únicamente la Web API del backend.

No se debe **MOSTRAR** imágenes para los productos a vender en el **FRONT-END WEB** porque no es un carrito de compras.



Objetivo general

Construir una aplicación web completa aplicando Clean Architecture, integrando backend, frontend, API REST, seguridad básica, control de stock, facturación e historial de ventas.

Requisitos obligatorios

Arquitectura

El sistema debe implementar Clean Architecture con al menos las siguientes capas:

- Domain
- Application
- Infrastructure
- API

Reglas obligatorias:

- Domain no debe depender de Infrastructure.
- Application define contratos.
- Infrastructure implementa persistencia.
- API no debe contener lógica de negocio.
- Frontend consume únicamente la API REST.

Modelo de datos mínimo obligatorio

El modelo debe incluir como mínimo:

- Product
- Customer
- Sale
- SaleDetail
- User
- Role
- StockMovement
- ErrorLog

Tablas

Productos

El sistema debe permitir:

- Crear productos;
- Listar productos;
- Actualizar productos;
- Activar/desactivar productos;
- Mostrar para la venta solo productos activos y con stock mayor que cero.



Clientes

El sistema debe permitir:

- Registrar clientes;
- Listar clientes;
- Consultar clientes;
- Activar/desactivar clientes.

Ventas

El sistema debe permitir crear una venta con encabezado y detalle.

La venta debe manejar al menos estos estados:

- Draft
- Confirmed
- Cancelled

Regla:

El stock se descuenta únicamente cuando la venta pasa a Confirmed.

Detalle de venta

El detalle debe permitir:

- Agregar productos;
- Quitar productos antes de confirmar;
- Cambiar producto antes de confirmar;
- No duplicar el mismo producto dentro de la misma venta;
- Validar que no se venda más cantidad que el stock disponible.

Historial de ventas

El sistema debe conservar el histórico de ventas.

Esto implica:

- No eliminar ventas confirmadas;

Regla:

Una factura antigua debe poder reconstruirse totalmente.

Seguridad

El sistema debe incluir:

- Login;
- JWT;
- Roles;



- mínimo dos roles:
 - Administradores (Administrators)
 - Vendedores (Sellers)
- Endpoints protegidos según rol.

El administrador puede:

- Administrar usuarios;
- Administrar roles;
- Administrar productos;
- Consultar todas las ventas;
- Desbloquear usuarios si se implementa bloqueo.
- Etc.
- Es decir, tiene acceso total al sistema menos a las opciones del vendedor.

El vendedor puede:

- Registrar ventas;
- Consultar productos;
- Consultar clientes (crear un cliente de forma directa en la pantalla de ventas cuando el cliente aún no está registrado);
- visualizar facturas del vendedor.

Validaciones

El sistema debe validar:

- Campos obligatorios;
- Formatos de correo;
- Identificación/cédula si aplica;
- Valores numéricos;
- Stock disponible;
- Productos duplicados en detalle;
- Datos requeridos para confirmar la venta.
- Etc.

Factura / PDF

El sistema debe permitir visualizar una venta/factura guardada (reconstrucción total de la venta/ factura).

La factura debe mostrar:

- Número de venta;
- Fecha;
- Cliente;
- Vendedor;
- Detalle de productos;
- Subtotal;
- IVA;
- Total.



Debe visualizarse en pantalla y exportarse a PDF.

Manejo de errores

El sistema debe mostrar mensajes claros al usuario.

Además, debe registrar errores relevantes en ErrorLog (Base de Datos).

Campos sugeridos (Pueden ser mas):

- Id
- Message
- ExceptionType
- StackTrace
- Source
- UserId
- CreatedAt

Requisitos opcionales (no son obligatorios)

El sistema puede permitir.

- Refresh tokens.
- Recuperación de contraseña.
- Autenticación externa.
- Auditoría general con AuditLog.
- Reportes avanzados.
- Exportación avanzada a PDF/Excel.
- Pruebas automatizadas.
- Despliegue completo frontend + backend + base de datos (esto puede ser en local) en la nube (**sin que esto implique costo alguno**).

Restricciones importantes

No se permite:

- Lógica de negocio en controllers (API);
- Lógica de negocio en repositorios;
- Acceso directo a Base de Datos (Objeto DbContext si se usa ASPNET Core) desde la API;
- CRUD libre sobre ventas confirmadas;
- Eliminar físicamente ventas confirmadas;
- Depender solo del frontend para seguridad.



Entregas parciales

Entrega 0 — Modelo de datos (SEMANA 1)

Debe incluir:

- Diagrama ER;
- Tablas;
- Relaciones;
- Justificación de históricos;
- Explicación de soft delete;
- Casos de validación del modelo.

Entrega 1 — Domain (SEMANA 1)

Debe incluir:

- Entidades;
- Reglas de negocio;
- Excepciones;
- Estados de venta;
- Pruebas unitarias básicas si aplica.

Entrega 2 — Application (SEMANA 2)

Debe incluir:

- Comandos;
- Queries;
- Handlers;
- Interfaces;
- Validaciones;
- DTOs.
- Mappers

Entrega 3 — Infrastructure (SEMANA 2)

Debe incluir:

- DbContext en el caso de .NET Core;
- Repositorios;
- Unit of Work;
- Migraciones en el caso de .NET Core;
- Persistencia.

Entrega 4 — API (SEMANA 3)

Debe incluir:

- Controllers;
- Endpoints;



- JWT;
- Roles;
- Manejo de errores.

Entrega 5 — Frontend (SEMANA 3)

Debe incluir:

- Login;
- Gestión de productos;
- Gestión de clientes;
- Creación de venta;
- Visualización de factura.

Entrega 6 — Defensa final (SEMANA 3)

Debe incluir:

- Explicación por capas;
- Justificación del modelo;
- Preguntas individuales.



A continuación, se explica con más detalle algunos de los requerimientos principales.

La aplicación debe tener la siguiente funcionalidad:

Nº	Requisito
1	Implemente un Modelo Entidad Relación que se ajuste al desarrollo utilizando principios de normalización. Se puede usar un modelo ya existente. El modelo debe considerar la seguridad para la administración de usuarios, roles de acceso a la aplicación, considerar mínimo 2 roles (Administradores, con acceso total al sistema. Rol de Vendedores, con acceso a realizar la venta y visualizar el catálogo de las ventas realizados por el vendedor. También se podría considerar en la ventana de ventas un botón para crear un cliente rápidamente en el caso que no se haya registrado aún el cliente, esto también debe estar en el CRUD de la tabla de clientes). Opcional: • recuperación de contraseña; • refresh tokens; • autenticación externa; • gestión avanzada de permisos.
2	Permitir crear una orden de venta con sus respectivos CAMPOS y TABLAS de forma lógica.
3	Un producto debería aparecer una sola vez en la orden . (No se debe permitir duplicarlo).
4	Calcular totales, subtotales e IVA (según sea necesario)
5	Editar SOLO los campos que sean necesarios, el resto bloquear (desactivar).
6	VALIDAR los campos para el ingreso de datos, esto es solo letras, solo números, no permitir valores en blanco, formatos como para correo electrónico, validar cédula, donde sea necesario, y todas las validaciones que sean lógicas y necesarias. Esto para todo el proyecto.
7	Tener en cuenta la disminución AUTOMÁTICA del inventario en la tabla correspondiente (productos).
8	Los productos para vender (a mostrar) deberían ser SOLO los que tienen stock mayor que cero y activos y no los que no tengan stock (stock en 0).
9	En la venta, no permitir vender más de lo que se tiene en stock de un producto seleccionado (validar).
10	Permitir eliminar un producto de la venta (orden de venta), esto lógicamente se debe realizar antes de almacenar (guardar) la orden de venta (Factura). Se debe confirmar la eliminación.
11	Permitir actualizar la cantidad a vender de un producto, sin perder el resto de los datos, esto se debe hacer antes de almacenar (guardar) la orden de venta (Factura) lógicamente.



12	Crear una opción en la página web (en el proyecto) que, dada una orden de venta realizada (guardada), muestre el encabezado y el detalle de la ORDEN DE VENTAS (Factura) creada. Es decir, reconstruir totalmente una factura y visualizar en PDF para su posterior descarga e impresión.
13	El diseño de la interfaz de usuario deberá cumplir estándares de usabilidad y facilidad de uso.
14	Control de Errores y mensajes personalizados de acuerdo con la situación. Almacenar los errores para una posterior revisión (por ejemplo, para dar soporte técnico).
15	Para buscar un cliente o un producto en la venta se debe realizar mediante una ventana emergente para una búsqueda inteligente (dinámica) que busque por todos los campos de la consulta (SELECT), además estas ventanas deben paginar por bloques para optimizar la recuperación de datos y permitir mediante un combobox seleccionar la cantidad de registros de la paginación (10, 15, 20, 30).
16	Utilizar buenas prácticas de programación y código limpio, principios SOLID y patrones de software donde sea requerido sin hacer sobre ingeniería. TODO EN INGLÉS y la GUI en español.
17	Utilizar transacciones (Se puede usar el patrón Unit Of Work) y conurrencia (optimista u pesimista) donde sea necesario según análisis. Mostrar los mensajes claros en el front-end de la transacción completada o fallida.
18	La primera pantalla debe ser una pantalla de validación (Login) que valide contra la base de datos, lo que debe pedir al usuario es el correo electrónico y la clave , la clave debe tener en cualquier orden, mínimo una letra mayúscula, una minúscula, un número y un carácter especial. La longitud de la clave debe ser de mínimo 8 caracteres y máximo 10) y una vez que se valide se ingresa al sistema de facturación. Opcional: Validación externa contra algún sistema de autenticación (como Google Login, Firebase Auth, Supabase Auth, Microsoft Account, etc.)
19	Dentro del sistema se debe tener la funcionalidad para que el administrador del sistema le permita administrar los roles y usuarios . Al crear un usuario de debe asignar a un rol específico (Rol de: Administradores o Vendedores), para que se pueda realizar la venta (Campos mínimos para la tabla de usuarios: nombre de usuario (user name), nombre, apellido, cédula (opcional), clave, confirmar clave, correo). NOTA: Si se usa como backend .NET se puede utilizar alguna librería para la gestión de la seguridad, por ejemplo, Identity de ASPNET Core. En el caso de otras tecnologías de backend como Node.js, PHP, Python, etc. También se puede usar librerías que faciliten la gestión de la seguridad de la aplicación.
20	Para la seguridad se debe usar JWT (JSON Web Token) con el propósito de transmitir información de forma segura y efectiva entre un determinado emisor y su respectivo receptor. JWT es utilizado por las aplicaciones para validar la identidad de un determinado usuario.
21	Es obligatorio realizar el mantenimiento (CRUD y búsquedas inteligentes por todos los campos específicos) de todas las tablas del modelo de datos en la cuales se deban hacer de forma lógica (revisar).



	Realizar mantenimiento, búsquedas inteligentes, paginación por todos los campos específicos de entidades maestras como productos, clientes, usuarios. Las entidades transaccionales como ventas, detalles de venta, registros de errores deben gestionarse mediante operaciones controladas, no mediante CRUD libre. NOTA: Una venta no se elimina, se anula/cancela Una producto y un cliente no se eliminan físicamente.
22	Realizar una programación por capas con Clean Architecture (No utilizar MVC como arquitectura). Se puede utilizar MVC como parte de Clean Architecture.
23	Para todo se debe usar servicios REST para trabajar con el BACK-END desde el FRONT-END WEB , es decir, crear una capa de servicios (REST) en la arquitectura por capas, para que el cliente web únicamente utilicen los servicios REST de la capa de servicios.
24	Opcional: Publicar la aplicación BACK-END, FRONT-END (y la base de datos alternativamente si se desea, aunque esta puede estar en local para un mayor rendimiento y por problemas de costo de publicación) en alguna plataforma Cloud (Puede ser en alguna plataforma libre o de prueba que no sea necesario pagar hasta revisar el trabajo).
25	Si el usuario digita de forma incorrecta los datos de ingreso en la pantalla de login por 3 veces consecutivas, el sistema debe bloquear el usuario correspondiente. El Administrador debe entonces poder ingresar al sistema y desbloquear al usuario bloqueado.
26	Los errores personalizados se deben almacenar en la base de datos con la información y datos necesarios y lógicos (Por ejemplo: usuario, error, línea del error, fecha, pantalla, evento, etc.) para una posterior revisión y dar el soporte técnico cuando el usuario lo requiera. Crear un reporte en el front-end para visualizar estos errores que deben ser accesibles solo a los administradores.
27	Crear un Script SQL o utilizar algún framework (fake data framework) que permita generar datos de prueba de por lo menos 100.000 registros en las tablas de clientes y productos y las tablas donde se guardan las ventas. Probar el rendimiento de la aplicación web al realizar o consultar las ventas realizadas o el resto de tablas. NOTA: Puesto que no hay servicios gratis que permitan almacenar la cantidad de datos requeridas (100.000 registros) se debe probar este punto en un servidor local (por facilidad es necesario que este requisito se implemente en otro servidor que no se haya publicado). Esto es un trabajo en grupo que se debe presentar antes del proyecto final. Se puede usar para esto alguno de los trabajos individuales ya presentados.
28	Los detalles de venta deben conservar datos históricos del producto vendido, como nombre, código, precio unitario, cantidad y subtotal. Es decir los datos necesarios que permitan reconstruir la venta.
29	Método de pago: Solo en efectivo
30	Requisito de eliminación



	El sistema debe permitir eliminar productos y clientes solo si no tienen ventas/pedidos asociados. Si ya tienen historial, no deben eliminarse físicamente; deben marcarse como inactivos o eliminados lógicamente.
31	El sistema debe registrar cada cambio de inventario en una tabla de movimientos de stock, indicando producto, tipo de movimiento, cantidad, stock anterior, stock nuevo, usuario responsable, fecha y referencia de la operación realizada.

SOFTWARE A UTILIZAR:

1. **BACK-END:** Tecnología a elección del grupo.
2. **FRONT-END:** Blazor, Angular, React, Vue.js, Svelte u otra tecnología de Front-End.

BASE DE DATOS (RELACIONAL): Cualquier base de datos.

NOTA: Para la **autenticación, autorización y notificaciones** se puede utilizar tecnologías como Firebase, Supabase u otras, siempre y cuando no haya que pagar.

Los requisitos marcados como OPCIONALES se consideran funcionalidades BONUS.

Estas funcionalidades pueden otorgar nota extra adicionales según:

- Complejidad;
- Correcta implementación;
- Calidad técnica;
- Integración arquitectónica;
- Defensa técnica individual.

La implementación de funcionalidades opcionales NO reemplaza requisitos obligatorios.



Detalles del sistema

Entidades mínimas:

- Product
- Customer
- Sale
- SaleDetail
- User
- Role
- StockMovement
- ErrorLog
- PaymentMethod

Opcional:

- Category
- Tax
- AuditLog

Estados de la venta

Agregar explícitamente:

- Draft / Borrador (venta en edición, es la venta que se está armando en caja, todavía no guardada formalmente) Puede existir solo en memoria del frontend o almacenarse temporalmente según diseño del grupo;
- Confirmed / Confirmada (venta finalizada y stock descontado);
- Cancelled / Anulada (venta anulada). Es una venta confirmada que luego se anula por alguna razón. Ejemplo: error de facturación, devolución, cancelación autorizada.

Esto como aclaración para saber cuándo se descuenta realmente el stock.

Regla clara de stock

El stock se descuenta únicamente cuando la venta se confirma.

Mientras la venta está en borrador o pendiente, no debería afectar inventario.

En un punto de venta, lo esencial es poder reconstruir:

- Qué se vendió;
- Cuándo;
- A quién;
- Quién vendió;
- A qué precio;
- Qué cantidad;
- Cuánto fue el total.



¿Qué es Soft Delete?

Soft Delete significa que el registro **no se elimina físicamente** de la base de datos, sino que se marca como eliminado o inactivo.

En sistemas reales, muchas veces no se eliminan datos... se desactivan.

¿Por qué NO eliminar directamente?

Integridad de la información

Ejemplo:

- Un cliente ya tiene ventas registradas
- Un producto ya fue vendido

Si se elimina:

- Se pierde historial
- Se rompe relaciones
- Genera inconsistencias

Auditoría (muy importante en sistemas reales)

Las empresas necesitan:

- Saber qué se vendió
- Quién compró
- Cuándo ocurrió

Eliminar datos rompe auditoría

Relaciones en base de datos

Ejemplo:

- Product → SaleDetail
- Customer → Sale

Si se elimina:

- Se puede romper claves foráneas
- O perder coherencia

Reglas de negocio

Ejemplo:

- Producto discontinuado
- Cliente inactivo

No se elimina, se **marca como inactivo**



SOLUCIÓN REAL

Soft Delete (el más usado)

¿Cuándo SÍ eliminar?

Casos válidos

- Datos de prueba
- Registros temporales
- Logs antiguos
- Información no relacionada

¿Cuándo NO eliminar?

- Clientes con historial
- Productos vendidos
- Ventas
- Pedidos

EN EL PROYECTO FINAL

Diseño recomendado

Product

- Crear
- Atualizar
- Activar/desactivar
- Eliminar

Customer

- Registrar
- Consultar
- Desactivar
- Eliminar

Sale / Order

- Crear
- Consultar
- Cancelar/Anular

Los datos importantes no se borran...
Se gestionan.



CONCLUSIÓN

- No eliminar es una decisión real de ingeniería
- Protege integridad, auditoría y consistencia
- Soft delete es el enfoque estándar

Diferencia entre Delete físico y Soft Delete

Tipo	Qué hace	Cuándo usar
Delete físico	Borra el registro de la BD	Datos nuevos sin historial
Soft Delete	Marca como eliminado	Datos con historial o relevancia

Soft Delete correctamente en Clean Architecture

La idea es esta:

Domain

Define el comportamiento de negocio:

Application

Decide si se puede eliminar:

Infrastructure

Solo ejecuta persistencia:

API

Solo recibe la petición:

Regla correcta para el proyecto final

Para productos y clientes:

Si no tienen ventas/pedidos asociados, se permite eliminación física.

Si tienen historial, se aplica soft delete o desactivación.